

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №4
по курсу «Программирование графических процессоров»

Работа с матрицам. Метод Гаусса.

Выполнил: И.А. Хомяков

Группа: 8О-407Б

Преподаватель: А.Ю. Морозов

Москва, 2024

Условие

1. Цель работы: Использование объединения запросов к глобальной памяти.
Реализация метода Гаусса с выбором главного элемента по столбцу.
Ознакомление с библиотекой алгоритмов для параллельных расчетов Thrust.
Использование двухмерной сетки потоков. Исследование производительности программы с помощью утилиты nvprof (обязательно отразить в отчете).
2. Вариант 3: Решение квадратной СЛАУ.

Программное и аппаратное обеспечение

Compute capability: 7.5

Name: Tesla T4

Total Global Memory: 15835660288

Shared memory per block: 49152

Registers per block: 65536

Warp size: 32

Max threads per block: (1024, 1024, 64)

Max block: (2147483647, 65535, 65535)

Total constant memory: 65536

Multiprocessors count: 40

Метод решения

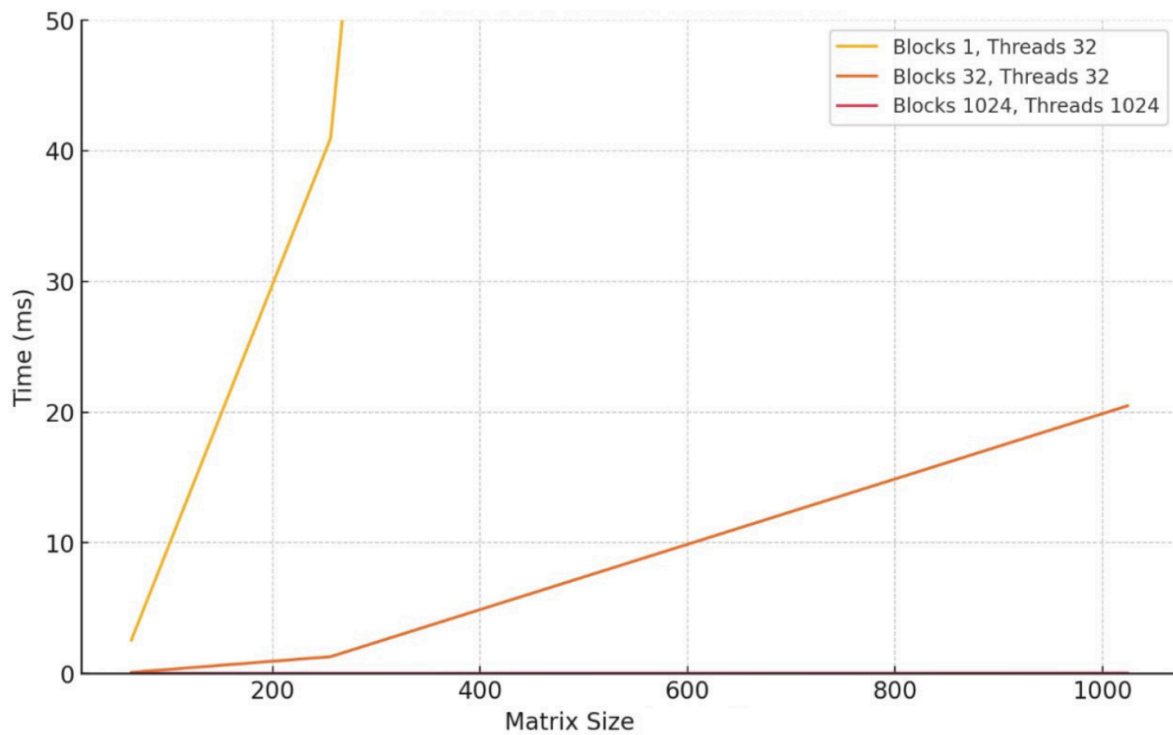
Есть СЛАУ вида $Ax = b$. Нам нужно создать расширенную матрицу $[A|b]$ и привести к верхнетреугольному виду. Пивотирование - выбор главного элемента. Для каждого столбца выбираем строку с максимальным по модулю элементом в текущем столбце. Это необходимо для численной стабильности метода. Далее меняем строки местами, чтобы главный элемент оказался на диагонали. Строка, содержащая главный элемент делится на этот элемент, чтобы сделать его равным единице. Из всех строк ниже текущей вычитается текущая, умноженная на соответствующий коэффициент, чтобы обнулить элементы под главным найденным. После приведения матрицы начинаем обратный ход для нахождения неизвестных. Начинаем с последнего уровня и подставляем неизвестные. Так продолжаем пока не найдем все неизвестные.

Описание программы

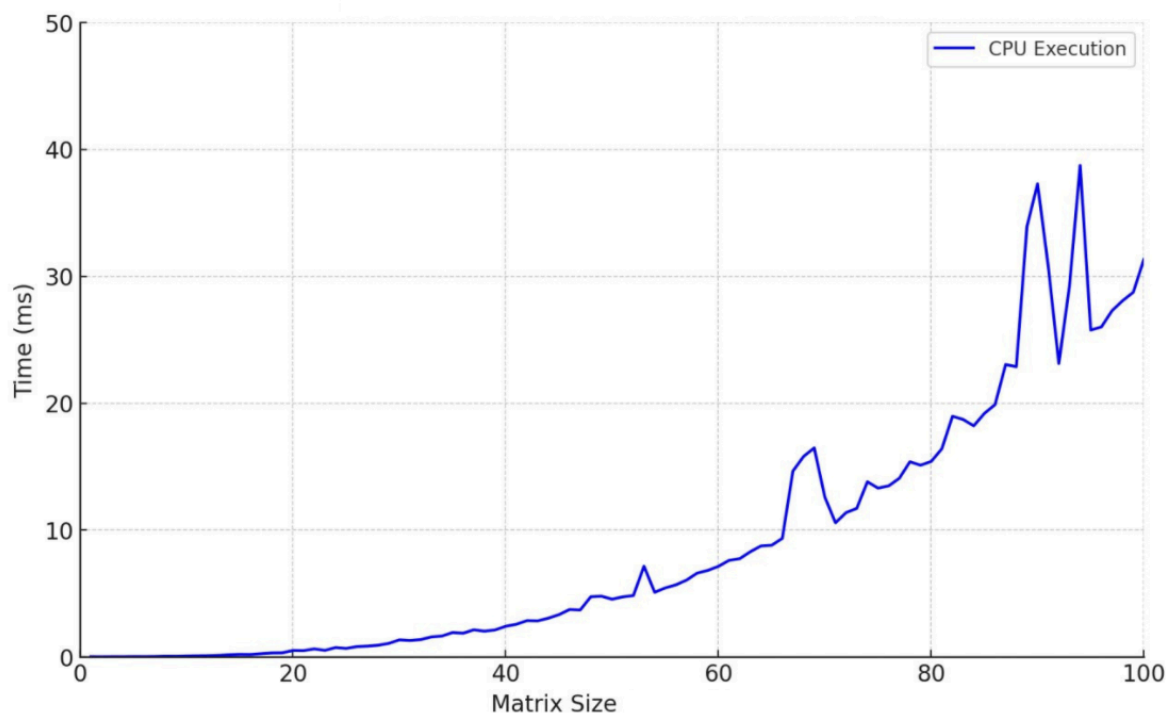
Программа из одного файла. Считываем матрицу и свободные коэффициенты, записываем их в массив. Вызываем solveGaussian, в ней копируем на gpu и для каждой колонки определяем startPtr и endPtr, чтобы через библиотеку thrust вызвать max_element и найти элемент с максимальным абсолютным значением. Передаем кастомный comparator, так как иначе max_element будет возвращать просто максимальный элемент. Далее вызываем ядро swapRows, чтобы поменять строку с максимальным элементом и текущую строку. Далее вызываем ядро gaussian, в котором вычитаем строки для устранения максимального элемента. Везде проверяем на 0 через eps, так как double нельзя сравнивать с 0 напрямую. Копируем приведенную матрицу обратно на host. Делаем обратный ход, вычисляя результат и выводим его.

Результаты

1. Графики зависимости времени выполнения от размера матрицы(GPU)



2. График времени выполнения на CPU



3. Профилирование с nvprof

Для матрицы 100 на 100 с параметрами `swapRows<<<1024, 32>>>` и `gaussian<<<dim3(32, 32), dim3(32, 32)>>>`

```
nvprof: NVIDIA (R) Cuda command line profiler
Copyright (c) 2012 - 2023 NVIDIA Corporation
Release version 12.2.142 (21)
==6084== NVPROF is profiling process 6084, command: ./lab_4
0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00
==6084== Profiling application: ./lab_4
==6084== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	70.22%	71.283ms	100	712.83us	536.12us	806.64us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_reduce::ReduceAgent<thrus
	13.79%	14.000ms	100	140.00us	139.87us	141.37us	gaussian(double*, int, int)
	5.69%	5.779ms	100	57.789us	57.502us	64.350us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
	5.32%	5.399ms	100	53.998us	53.471us	54.783us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
	4.79%	4.867ms	100	48.670us	48.095us	49.535us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
	0.17%	171.16us	101	1.6940us	1.5990us	8.2560us	[CUDA memcpY DtoH]
	0.01%	9.6320us	1	9.6320us	9.6320us	9.6320us	[CUDA memcpY HtoD]
	45.68%	180.99ms	201	900.44us	2.8560us	180.10ms	cudaMalloc
	29.59%	117.23ms	500	234.46us	1.2070us	19.637ms	cudaStreamSynchronize
	17.78%	70.444ms	1	70.444ms	70.444ms	70.444ms	cudaFuncGetAttributes
API calls:	3.64%	14.417ms	100	144.17us	142.62us	157.16us	cudaDeviceSynchronize
	2.21%	8.745ms	500	17.490us	3.9490us	4.4353ms	cudaLaunchKernel
	0.36%	1.445ms	100	14.453us	11.964us	25.514us	cudaMemcpyAsync
	0.24%	969.32us	201	4.8220us	2.0570us	216.85us	cudaFree
	0.20%	778.65us	4903	158ns	113ns	5.8000us	cudaGetLastError
	0.11%	431.99us	1101	392ns	279ns	3.5470us	cudaGetDevice
	0.06%	250.85us	600	418ns	281ns	2.8810us	cudaDeviceGetAttribute
	0.05%	184.75us	2	92.376us	53.367us	131.39us	cudaMemcpy
	0.03%	136.67us	800	170ns	119ns	567ns	cudaPeekAtLastError
	0.03%	135.95us	114	1.1920us	133ns	51.332us	cuDeviceGetAttribute
	0.00%	10.969us	1	10.969us	10.969us	10.969us	cuDeviceGetName
	0.00%	5.5130us	1	5.5130us	5.5130us	5.5130us	cuDeviceGetPCIBusId
	0.00%	4.5820us	1	4.5820us	4.5820us	4.5820us	cuDeviceTotalMem
	0.00%	1.5260us	3	508ns	213ns	1.0340us	cuDeviceGetCount
	0.00%	975ns	2	487ns	194ns	781ns	cuDeviceGet
	0.00%	480ns	1	480ns	480ns	480ns	cuModuleGetLoadingMode
	0.00%	331ns	1	331ns	331ns	331ns	cudaGetDeviceCount
	0.00%	237ns	1	237ns	237ns	237ns	cuDeviceGetUuid

Для матрицы 1000 на 1000

```
nvprof: NVIDIA (R) Cuda command line profiler
Copyright (c) 2012 - 2023 NVIDIA Corporation
Release version 12.2.142 (21)
==6474== NVPROF is profiling process 6474, command: ./lab_4
0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00
==6474== Profiling application: ./lab_4
==6474== Profiling result:
```

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU activities:	80.34%	599.32ms	1000	599.32us	205.56us	1.7705ms	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_reduce::ReduceAgent<thrus
	8.41%	62.719ms	1000	62.719us	52.318us	141.05us	gaussian(double*, int, int)
	3.50%	26.076ms	1000	26.075us	21.695us	63.071us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
	3.25%	24.242ms	1000	24.242us	19.872us	54.431us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
	2.95%	21.980ms	1000	21.979us	18.079us	49.375us	void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
	1.00%	7.4906ms	1001	7.4830us	1.1520us	6.2166ms	[CUDA memcpY DtoH]
	0.56%	4.1544ms	1	4.1544ms	4.1544ms	4.1544ms	[CUDA memcpY HtoD]
	53.51%	685.31ms	5000	137.06us	1.1880us	4.5093ms	cudaStreamSynchronize
	22.41%	286.96ms	2001	143.41us	3.2660us	187.32ms	cudaMalloc
	6.36%	81.464ms	2001	40.711us	3.9080us	474.40us	cudaFree
API calls:	5.22%	66.842ms	1	66.842ms	66.842ms	66.842ms	cudaFuncGetAttributes
	5.22%	66.807ms	1000	66.806us	32.153us	1.2641ms	cudaDeviceSynchronize
	3.29%	42.081ms	5000	8.4160us	3.9290us	1.2736ms	cudaLaunchKernel
	1.31%	16.799ms	1000	16.798us	12.810us	159.45us	cudaMemcpyAsync
	0.99%	12.667ms	2	6.3336ms	4.5818ms	8.1654ms	cudaMemcpy
	0.83%	10.616ms	49003	216ns	113ns	500.11us	cudaGetLastError
	0.43%	5.5344ms	11001	503ns	276ns	20.756us	cudaGetDevice
	0.26%	3.2791ms	6000	546ns	270ns	14.986us	cudaDeviceGetAttribute
	0.17%	2.2254ms	8000	278ns	116ns	569.32us	cudaPeekAtLastError
	0.01%	132.25us	114	1.1600us	143ns	50.839us	cuDeviceGetAttribute
	0.00%	11.581us	1	11.581us	11.581us	11.581us	cuDeviceGetName
	0.00%	5.2500us	1	5.2500us	5.2500us	5.2500us	cuDeviceTotalMem
	0.00%	5.2320us	1	5.2320us	5.2320us	5.2320us	cuDeviceGetPCIBusId
	0.00%	1.2500us	3	416ns	196ns	776ns	cuDeviceGetCount
	0.00%	1.0530us	2	526ns	196ns	857ns	cuDeviceGet
	0.00%	668ns	1	668ns	668ns	668ns	cuModuleGetLoadingMode
	0.00%	377ns	1	377ns	377ns	377ns	cuDeviceGetUuid
	0.00%	211ns	1	211ns	211ns	211ns	cudaGetDeviceCount

Для матрицы 5000 на 5000

```

17 nvprof: NVIDIA (R) Cuda command line profiler
   Copyright (c) 2012 - 2023 NVIDIA Corporation
   Release version 12.2.142 (21)
   ==7616== NVPROF is profiling process 7616, command: ./lab_4
   0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00 0.000000000e+00
   ==7616== Profiling application: ./lab_4
   ==7616== Profiling result:
            Type Time(%)      Time       Calls      Avg       Min       Max      Name
GPU activities: 57.29%    2.82371s    3720    759.06us    719.25us    1.9528ms    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::__reduce::ReduceAgent<thrus
13.78%    679.10ms    1200    530.54us    530.54us    743.63us    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::__reduce::ReduceAgent<thrus
10.33%    509.33ms    3720    136.91us    118.72us    366.49us    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::__reduce::ReduceAgent<thrus
5.52%    271.95ms    5000    54.389us    52.350us    140.80us    gaussian(double*, int, int)
4.43%    218.17ms    5001    43.625us    1.1510us    212.13ms    [CUDA memory DtoH]
2.52%    124.07ms    5000    24.814us    21.696us    62.175us    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
2.14%    105.70ms    5000    21.139us    19.903us    54.878us    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
1.94%    95.805ms    5000    19.160us    18.048us    49.439us    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::_parallel_for::ParallelFor
1.83%    90.266ms    1    90.266ms    90.266ms    90.266ms    [CUDA memcopy HtoD]
0.21%    10.402ms    3720    2.7960us    2.5280us    7.1050us    void thrust::cuda_cub::core::_kernel_agent<thrust::cuda_cub::__reduce::DrainAgent<int>,
API calls: 62.46%    4.86236s    25000    194.49us    1.4680us    12.432ms    cudaStreamSynchronize
11.84%    321.78ms    10001    92.168us    3.1110us    216.81ms    cudaMalloc
6.84%    532.16ms    10001    53.211us    3.6740us    9.1274ms    cudaFree
4.83%    376.06ms    32440    11.592us    3.7230us    10.537ms    cudaLaunchKernel
4.28%    331.19ms    5000    66.638us    21.530us    18.043ms    cudaDeviceSynchronize
3.91%    304.66ms    2    152.33ms    90.787ms    213.87ms    cudaMemcopy
1.72%    133.84ms    319403    419ns    112ns    9.0178ms    cudaGetLastError
1.23%    96.084ms    5000    19.216us    12.556us    8.0947ms    cudaMemcopyAsync
1.12%    86.941ms    1    86.941ms    86.941ms    86.941ms    cudaFuncGetAttributes
0.56%    43.939ms    69881    628ns    275ns    559.93us    cudaGetDevice
0.54%    42.146ms    44880    939ns    269ns    7.5778ms    cudaDeviceGetAttribute
0.36%    27.744ms    54880    505ns    112ns    13.848ms    cudaPeekAtLastError
0.30%    23.233ms    7440    3.1220us    713ns    18.458ms    cudaOccupancyMaxActiveBlocksPerMultiprocessorWithFlags
0.00%    207.37us    114    1.8190us    201ns    73.079us    cudaDeviceGetAttribute
0.00%    14.742us    1    14.742us    14.742us    14.742us    cuDeviceGetName
0.00%    8.4330us    1    8.4330us    8.4330us    8.4330us    cuDeviceGetPCIBusId
0.00%    6.3620us    1    6.3620us    6.3620us    6.3620us    cuDeviceTotalMem
0.00%    2.0980us    3    699ns    295ns    1.4660us    cuDeviceGetCount
0.00%    1.1030us    2    551ns    268ns    835ns    cuDeviceGet

```

Видно, что большую часть времени занимают операции thrust, но при увеличении количества данных занимаемый процент уменьшается. Скорее всего связано с тем, что под копотом thrust автоматически подбирает оптимальную параллелизацию под размер входных данных. Так как параллелизация ядер в коде не поменялась.

Выводы

Данный алгоритм можно применять для решения СЛАУ. Чаще всего используется в моделировании физических процессов, решении задач оптимизации, симуляции систем реального времени и для обработки данных в машинном обучении. Благодаря насмотренности и опыту поиска проблем в 5-ой лабораторной работе с алгоритмом `blelloch` было относительно несложно разобраться в том, как распараллелить алгоритм Гаусса. Также помогли примеры и наставления с лекции.

Были две проблемы: одна из них с проверкой на нули. Если ее не делать, то решение может стать NAN или INF. Вторая была связана с опечаткой при запуске ядра swar: запускал двумерную сетку для решения одномерной задачи.

За счет распараллеливания на g pi алгоритм выполняется куда быстрее, чем на cpu . Это сразу заметно даже на небольших данных, так как у алгоритма Гауса кубическая асимптотическая сложность.